

Chapter 5: R Objects

Hands-On Programming with R — Sections 5.1, 5.3, 5.4, 5.7, and 5.8

Kalpana Renjarla

July 19, 2026

Contents

Introduction	1
Section 5.1 — Atomic Vectors	2
5.1.1 Doubles	3
5.1.2 Integers	3
5.1.3 Characters	3
5.1.4 Logicals	4
5.1.5 Complex and Raw	5
Section 5.3 — Matrices	5
Section 5.4 — Arrays	7
Section 5.7 — Lists	8
Section 5.8 — Data Frames	9
Complete 52-Card Data Frame	10
Conclusion	12
Reference	12

```
# Show all R code in the knitted document, as required by the assignment.  
knitr::opts_chunk$set(echo = TRUE, collapse = FALSE, comment = "##")
```

Introduction

This documented R Markdown program replicates and executes every line of code in Sections 5.1, 5.3, 5.4, 5.7, and 5.8 of Garrett Golemund's *Hands-On Programming with R*. Comments explain the purpose of each operation. The selected sections do not create graphs or charts, so graph interpretation is not applicable.

Section 5.1 — Atomic Vectors

Atomic vectors are one-dimensional objects whose elements all share one data type. This section demonstrates R's six atomic-vector types: double, integer, character, logical, complex, and raw.

```
# Create a numeric atomic vector representing the six faces of a die.
```

```
die <- c(1, 2, 3, 4, 5, 6)
```

```
# Display the vector.
```

```
die
```

```
## [1] 1 2 3 4 5 6
```

```
# Confirm that die is a vector.
```

```
is.vector(die)
```

```
## [1] TRUE
```

```
# Create and display a one-element vector.
```

```
five <- 5
```

```
five
```

```
## [1] 5
```

```
# Confirm its vector status and compare vector lengths.
```

```
is.vector(five)
```

```
## [1] TRUE
```

```
length(five)
```

```
## [1] 1
```

```
length(die)
```

```
## [1] 6
```

```
# Create one-element integer and character vectors.
```

```
int <- 1L
```

```
text <- "ace"
```

```
# Create multi-element integer and character vectors.
```

```
int <- c(1L, 5L)
```

```
text <- c("ace", "hearts")
```

```
# Sum works with the integer vector.
```

```
sum(int)
```

```
## [1] 6
```

The next line is intentionally reproduced from the book. It demonstrates that `sum()` cannot add character strings. The chunk permits the expected error so knitting can continue.

```
# Expected error: character strings cannot be summed.
sum(text)
```

```
## Error in 'sum()':
## ! invalid 'type' (character) of argument
```

5.1.1 Doubles

```
# Recreate the die; ordinary numeric literals are stored as doubles.
die <- c(1, 2, 3, 4, 5, 6)
die
```

```
## [1] 1 2 3 4 5 6
```

```
# Identify the atomic type.
typeof(die)
```

```
## [1] "double"
```

5.1.2 Integers

```
# The L suffix explicitly creates integer values.
int <- c(-1L, 2L, 4L)
int
```

```
## [1] -1 2 4
```

```
typeof(int)
```

```
## [1] "integer"
```

```
# Demonstrate a small floating-point rounding error with doubles.
sqrt(2)^2 - 2
```

```
## [1] 4.440892e-16
```

5.1.3 Characters

```
# Create and inspect a character vector.
text <- c("Hello", "World")
text
```

```
## [1] "Hello" "World"
```

```
typeof(text)
```

```
## [1] "character"
```

```
# A single quoted word is also a character vector of length one.
```

```
typeof("Hello")
```

```
## [1] "character"
```

Exercise 5.1 — Character or Number?

Question: Which of 1, "1", and "one" are character strings, and which are numbers?

Answer: 1 is numeric. "1" and "one" are character strings because quotation marks cause R to store their contents as character data.

```
# Verify the answer by inspecting each value's type.
```

```
typeof(1)
```

```
## [1] "double"
```

```
typeof("1")
```

```
## [1] "character"
```

```
typeof("one")
```

```
## [1] "character"
```

5.1.4 Logicals

```
# Evaluate a comparison; 3 is not greater than 4.
```

```
3 > 4
```

```
## [1] FALSE
```

```
# Create and inspect a logical vector.
```

```
logic <- c(TRUE, FALSE, TRUE)
```

```
logic
```

```
## [1] TRUE FALSE TRUE
```

```
typeof(logic)
```

```
## [1] "logical"
```

```
# F is R's shorthand for FALSE unless F has been reassigned.  
typeof(F)
```

```
## [1] "logical"
```

5.1.5 Complex and Raw

```
# Create and inspect complex numbers containing imaginary components.  
comp <- c(1 + 1i, 1 + 2i, 1 + 3i)  
comp
```

```
## [1] 1+1i 1+2i 1+3i
```

```
typeof(comp)
```

```
## [1] "complex"
```

```
# Create three empty raw bytes and inspect their type.  
raw(3)
```

```
## [1] 00 00 00
```

```
typeof(raw(3))
```

```
## [1] "raw"
```

Exercise 5.2 — Vector of Cards

Question: Create an atomic vector that stores the face names of the five cards in a royal flush. Which vector type is appropriate?

Answer: A character vector is appropriate because card faces are text labels.

```
# Store the five royal-flush face names in a character vector.  
hand <- c("ace", "king", "queen", "jack", "ten")  
hand
```

```
## [1] "ace" "king" "queen" "jack" "ten"
```

```
# Confirm that the object stores character data.  
typeof(hand)
```

```
## [1] "character"
```

Section 5.3 — Matrices

Matrices arrange one atomic vector into two dimensions. Because matrices are built from atomic vectors, all cells retain the same underlying data type.

```
# Arrange die into two rows; R fills columns by default.
m <- matrix(die, nrow = 2)
m
```

```
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
```

```
# Arrange the same values by rows instead of columns.
m <- matrix(die, nrow = 2, byrow = TRUE)
m
```

```
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
```

Exercise 5.3 — Make a Matrix

Question: Create a five-row, two-column matrix containing the face and suit of every card in a spade royal flush.

Answer: The book demonstrates several equivalent constructions.

```
# List all faces first and all suits second.
hand1 <- c("ace", "king", "queen", "jack", "ten", "spades", "spades",
           "spades", "spades", "spades")

# Method 1: specify five rows.
matrix(hand1, nrow = 5)
```

```
##      [,1] [,2]
## [1,] "ace" "spades"
## [2,] "king" "spades"
## [3,] "queen" "spades"
## [4,] "jack" "spades"
## [5,] "ten" "spades"
```

```
# Method 2: specify two columns.
matrix(hand1, ncol = 2)
```

```
##      [,1] [,2]
## [1,] "ace" "spades"
## [2,] "king" "spades"
## [3,] "queen" "spades"
## [4,] "jack" "spades"
## [5,] "ten" "spades"
```

```
# Method 3: directly assign the dimensions attribute.
dim(hand1) <- c(5, 2)
hand1
```

```
##      [,1]      [,2]
## [1,] "ace"     "spades"
## [2,] "king"    "spades"
## [3,] "queen"   "spades"
## [4,] "jack"    "spades"
## [5,] "ten"     "spades"

# Alternatively, list each face beside its suit.
hand2 <- c("ace", "spades", "king", "spades", "queen", "spades", "jack",
           "spades", "ten", "spades")

# Fill row by row while specifying either rows or columns.
matrix(hand2, nrow = 5, byrow = TRUE)
```

```
##      [,1]      [,2]
## [1,] "ace"     "spades"
## [2,] "king"    "spades"
## [3,] "queen"   "spades"
## [4,] "jack"    "spades"
## [5,] "ten"     "spades"

matrix(hand2, ncol = 2, byrow = TRUE)
```

```
##      [,1]      [,2]
## [1,] "ace"     "spades"
## [2,] "king"    "spades"
## [3,] "queen"   "spades"
## [4,] "jack"    "spades"
## [5,] "ten"     "spades"
```

Section 5.4 — Arrays

Arrays extend matrices to three or more dimensions. The example creates three 2-by-2 slices.

```
# Combine three number ranges and arrange them into a 2 x 2 x 3 array.
ar <- array(c(11:14, 21:24, 31:34), dim = c(2, 2, 3))

# Display all three array slices.
ar
```

```
## , , 1
##
##      [,1] [,2]
## [1,]  11  13
## [2,]  12  14
##
## , , 2
##
##      [,1] [,2]
## [1,]  21  23
## [2,]  22  24
```

```
##
## , , 3
##
##      [,1] [,2]
## [1,]   31   33
## [2,]   32   34
```

Interpretation: Slice 1 contains values 11–14, slice 2 contains 21–24, and slice 3 contains 31–34. R fills each slice by columns.

Section 5.7 — Lists

Unlike an atomic vector, a list can group R objects of different types and lengths, including another list.

```
# Create a list containing a numeric sequence, text, and a nested logical list.
list1 <- list(100:130, "R", list(TRUE, FALSE))

# Display the nested list structure.
list1

## [[1]]
## [1] 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118
## [20] 119 120 121 122 123 124 125 126 127 128 129 130
##
## [[2]]
## [1] "R"
##
## [[3]]
## [[3]][[1]]
## [1] TRUE
##
## [[3]][[2]]
## [1] FALSE
```

Exercise 5.5 — Use a List to Make a Card

Question: Use a list to store the face, suit, and point value of the ace of hearts.

Answer: A list preserves the face and suit as character values while keeping the point value numeric.

```
# Store three differently typed properties in separate list elements.
card <- list("ace", "hearts", 1)
card

## [[1]]
## [1] "ace"
##
## [[2]]
## [1] "hearts"
##
## [[3]]
## [1] 1
```


Section 5.8 — Data Frames

A data frame is a two-dimensional list. Each column is a vector, columns can have different types, and all columns must have equal lengths.

```
# Build a three-row data frame with face, suit, and numeric value columns.
df <- data.frame(face = c("ace", "two", "six"),
  suit = c("clubs", "clubs", "clubs"), value = c(1, 2, 3))

# Display the data frame.
df
```

```
##   face suit value
## 1  ace clubs     1
## 2  two clubs     2
## 3  six clubs     3
```

```
# Demonstrate named elements in a list and vector.
list(face = "ace", suit = "hearts", value = 1)
```

```
## $face
## [1] "ace"
##
## $suit
## [1] "hearts"
##
## $value
## [1] 1
```

```
c(face = "ace", suit = "hearts", value = "one")
```

```
##   face    suit    value
##   "ace" "hearts"  "one"
```

```
# Verify that a data frame is a list with class data.frame.
typeof(df)
```

```
## [1] "list"
```

```
class(df)
```

```
## [1] "data.frame"
```

```
# Display the internal structure of each column.
str(df)
```

```
## 'data.frame':   3 obs. of  3 variables:
## $ face : chr  "ace" "two" "six"
## $ suit : chr  "clubs" "clubs" "clubs"
## $ value: num  1 2 3
```

Version note: The book was written when `data.frame()` converted text columns to factors by default. Current R versions default to character columns, so `str(df)` may display `chr` instead of the factor output printed in the book. This is an expected version difference, not an error.

```
# Explicitly prevent conversion of character strings to factors.
df <- data.frame(face = c("ace", "two", "six"),
  suit = c("clubs", "clubs", "clubs"), value = c(1, 2, 3),
  stringsAsFactors = FALSE)

# Verify the resulting data frame and its column types.
df
```

```
##   face suit value
## 1  ace clubs     1
## 2  two clubs     2
## 3  six clubs     3
```

```
str(df)
```

```
## 'data.frame':   3 obs. of  3 variables:
## $ face : chr  "ace" "two" "six"
## $ suit : chr  "clubs" "clubs" "clubs"
## $ value: num  1 2 3
```

Complete 52-Card Data Frame

```
# Construct the complete playing-card deck exactly as shown in Section 5.8.
deck <- data.frame(
  face = c("king", "queen", "jack", "ten", "nine", "eight", "seven", "six",
    "five", "four", "three", "two", "ace", "king", "queen", "jack", "ten",
    "nine", "eight", "seven", "six", "five", "four", "three", "two", "ace",
    "king", "queen", "jack", "ten", "nine", "eight", "seven", "six", "five",
    "four", "three", "two", "ace", "king", "queen", "jack", "ten", "nine",
    "eight", "seven", "six", "five", "four", "three", "two", "ace"),
  suit = c("spades", "spades", "spades", "spades", "spades", "spades",
    "spades", "spades", "spades", "spades", "spades", "spades", "spades",
    "clubs", "clubs", "clubs", "clubs", "clubs", "clubs", "clubs", "clubs",
    "clubs", "clubs", "clubs", "clubs", "clubs", "diamonds", "diamonds",
    "diamonds", "diamonds", "diamonds", "diamonds", "diamonds", "diamonds",
    "diamonds", "diamonds", "diamonds", "diamonds", "diamonds", "hearts",
    "hearts", "hearts", "hearts", "hearts", "hearts", "hearts", "hearts",
    "hearts", "hearts", "hearts", "hearts", "hearts"),
  value = c(13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 13, 12, 11, 10, 9, 8,
    7, 6, 5, 4, 3, 2, 1, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 13, 12, 11,
    10, 9, 8, 7, 6, 5, 4, 3, 2, 1)
)

# Verify the completed deck and its dimensions.
deck
```

```
##   face      suit value
```

## 1	king	spades	13
## 2	queen	spades	12
## 3	jack	spades	11
## 4	ten	spades	10
## 5	nine	spades	9
## 6	eight	spades	8
## 7	seven	spades	7
## 8	six	spades	6
## 9	five	spades	5
## 10	four	spades	4
## 11	three	spades	3
## 12	two	spades	2
## 13	ace	spades	1
## 14	king	clubs	13
## 15	queen	clubs	12
## 16	jack	clubs	11
## 17	ten	clubs	10
## 18	nine	clubs	9
## 19	eight	clubs	8
## 20	seven	clubs	7
## 21	six	clubs	6
## 22	five	clubs	5
## 23	four	clubs	4
## 24	three	clubs	3
## 25	two	clubs	2
## 26	ace	clubs	1
## 27	king	diamonds	13
## 28	queen	diamonds	12
## 29	jack	diamonds	11
## 30	ten	diamonds	10
## 31	nine	diamonds	9
## 32	eight	diamonds	8
## 33	seven	diamonds	7
## 34	six	diamonds	6
## 35	five	diamonds	5
## 36	four	diamonds	4
## 37	three	diamonds	3
## 38	two	diamonds	2
## 39	ace	diamonds	1
## 40	king	hearts	13
## 41	queen	hearts	12
## 42	jack	hearts	11
## 43	ten	hearts	10
## 44	nine	hearts	9
## 45	eight	hearts	8
## 46	seven	hearts	7
## 47	six	hearts	6
## 48	five	hearts	5
## 49	four	hearts	4
## 50	three	hearts	3
## 51	two	hearts	2
## 52	ace	hearts	1

```
dim(deck)
```

```
## [1] 52 3
```

```
str(deck)
```

```
## 'data.frame': 52 obs. of 3 variables:  
## $ face : chr "king" "queen" "jack" "ten" ...  
## $ suit : chr "spades" "spades" "spades" "spades" ...  
## $ value: num 13 12 11 10 9 8 7 6 5 4 ...
```

Interpretation: The result contains 52 observations (one card per row) and three variables: **face**, **suit**, and **value**. The face and suit columns contain text, while value contains numeric point values.

Conclusion

The replicated code demonstrates how R builds increasingly flexible objects: atomic vectors store one data type, matrices and arrays organize atomic data across dimensions, lists combine objects of different types, and data frames organize equal-length vectors as analysis-ready columns. All requested code from Sections 5.1, 5.3, 5.4, 5.7, and 5.8 has been documented and included.

Reference

Grolemund, G. (2014). *Hands-on programming with R*. O'Reilly Media. <https://rstudio-education.github.io/hopr/r-objects.html>